

En **JDBC (Java Database Connectivity)**, lo esencial es entender **qué clases e interfaces son más usadas y qué métodos principales tiene cada una** para manejar un CRUD con bases de datos como MySQL.

Principales clases e interfaces de JDBC

Están en el paquete `java.sql`:

1. **DriverManager**

- Se encarga de gestionar los *drivers* y establecer conexiones.
- Método más usado:
 - `getConnection(url, user, password)` → devuelve un objeto `Connection`.

2. **Connection (interfaz)**

- Representa la conexión activa a la base de datos.
- Métodos importantes:
 - `createStatement()` → devuelve un `Statement`.
 - `prepareStatement(sql)` → devuelve un `PreparedStatement`.
 - `close()` → cierra la conexión.
 - `setAutoCommit(false)` y `commit()` / `rollback()` → control de transacciones.

3. **PreparedStatement (subinterfaz de Statement)**

- Para consultas con parámetros (más seguro y eficiente que `Statement`).
- Métodos principales:

- `setString(pos, valor)`, `setInt(pos, valor)`, etc. → asignar parámetros.
- `executeQuery()` → devuelve un `ResultSet`.
- `executeUpdate()` → devuelve número de filas afectadas.
- `close()`.

4. `ResultSet` (interfaz)

- Representa el resultado de una consulta (`SELECT`).
- Métodos principales:
 - `next()` → avanza al siguiente registro (true si hay más).
 - `getInt(columna)`, `getString(columna)`, etc. → obtener valores por nombre o índice.
 - `close()`.

5. `SQLException`

- Excepción que captura errores de JDBC.
- Métodos útiles:
 - `getMessage()` → descripción del error.
 - `getSQLState()` → código del estado SQL.
 - `getErrorCode()` → código específico del motor de BD.

Alta típica en JDBC

```
try {
    // 1. Conexión
    Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/empresa", "root", "tu_password");

    // 2. Sentencia preparada
    String sql = "INSERT INTO clientes (nombre, telefono, direccion) VALUES (?, ?, ?)";
    PreparedStatement stmt = conn.prepareStatement(sql);
    stmt.setString(1, "Juan Pérez");
```

```
stmt.setString(2, "123456789");
stmt.setString(3, "Calle Falsa 123");

// 3. Ejecutar
int filas = stmt.executeUpdate();
System.out.println("Filas insertadas: " + filas);

// 4. Cerrar
stmt.close();
conn.close();

} catch (SQLException e) {
    System.out.println("Error SQL: " + e.getMessage());
}
```

En resumen:

- **DriverManager** → abre conexión.
- **Connection** → maneja la sesión.
- **Statement / PreparedStatement** → ejecuta SQL.
- **ResultSet** → lee resultados.
- **SQLException** → maneja errores.

tabla comparativa con los métodos más usados de cada clase

Chuletila JDBC en Java

Clase / Interfaz	Uso principal	Métodos clave
DriverManager	Gestionar drivers y abrir conexión	<code>getConnection(url, user, pass)</code>
Connection	Representa la conexión con la BD	<code>createStatement()</code> → devuelve <code>Statement</code> <code>prepareStatement(sql)</code> → devuelve <code>PreparedStatement</code> <code>setAutoCommit(false)</code> → desactiva autocommit <code>commit()</code> / <code>rollback()</code> → control transacciones <code>close()</code> → cerrar conexión
Statement	Ejecutar SQL simple (sin parámetros)	<code>executeQuery(sql)</code> → <code>SELECT</code> → <code>ResultSet</code> <code>executeUpdate(sql)</code> → <code>INSERT/UPDATE/DELETE</code> (devuelve nº filas) <code>close()</code>
PreparedStatement	Ejecutar SQL con parámetros (seguro y rápido)	<code>setString(pos, valor)</code> <code>setInt(pos, valor)</code> <code>setDouble(pos, valor)</code>

		<code>executeQuery()</code> → SELECT <code>executeUpdate()</code> → INSERT/UPDATE/DELETE <code>close()</code>
ResultSet	Recorrer resultados de consultas	<code>next()</code> → avanza a la siguiente fila <code>getInt(columna)</code> <code>getString(columna)</code> <code>getDouble(columna)</code> <code>close()</code>
SQLException	Manejar errores de SQL	<code>getMessage()</code> → descripción <code>getSQLState()</code> → código SQL estándar <code>getErrorCode()</code> → código propio del motor BD

Consulta típica con JDBC

```
try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    String sql = "SELECT * FROM clientes WHERE codigo = ?";
    PreparedStatement stmt = conn.prepareStatement(sql);
    stmt.setInt(1, 33); // primer parámetro = 33

    ResultSet rs = stmt.executeQuery();
    while (rs.next()) {
        System.out.println(rs.getInt("codigo") + " - " +
            rs.getString("nombre") + " - " +
            rs.getString("telefono"));
    }

    rs.close();
    stmt.close();
} catch (SQLException e) {
```

```
    System.out.println("Error: " + e.getMessage());  
}
```